JavaScript Research

Variables, Comparisons, and Data Types

Main reference: MDN Web Docs

Topic: How JavaScript handles variables, comparisons, and different

data types\

Main reference: MDN Web Docs

1. What are the differences between `let`, `const`, and `var`? When should you use each one?

JavaScript has three common keywords for declaring variables: `let`,

`const`, and `var`. They differ in whether a variable can be reassigned

and where it is available in the code.

block-scoped, meaning it is available only inside the block

(`{ ... }`) where it is declared.

reassigned. It must receive an initial value when declared. If it

contains an object or array, the object's contents can still be

changed even though the variable cannot be reassigned.

redeclared in the same scope. These behaviors can make code harder

to understand, so it is usually avoided in modern JavaScript.

• **`let`** declares a variable that can be reassigned. It is

• **`const`** declares a block-scoped variable that cannot be

• **`var`** is function-scoped rather than block-scoped, and it can be

When to use them: Prefer `const` when you do not need to reassign a

variable. Use `let` when its value needs to change. Use `var` mainly

when reading or maintaining older code.

Example

```
let score = 10;
score = 15;

const country = "Egypt";

var oldStyle = "legacy";
oldStyle = "updated";

console.log(score);
console.log(country);
console.log(oldStyle);

if (true) {
    let blockValue = "inside block";
    var functionValue = "visible outside the block";
    console.log(blockValue);
}
```

```
console.log(functionValue);
// console.log(blockValue); // Uncommenting this line causes a ReferenceError.
```

Expected console output

```
15
Egypt
updated
inside block
visible outside the block
```

What this shows: `score` and `oldStyle` can be reassigned. `country` cannot be reassigned. The `let` variable `blockValue` is limited to the `if` block, while the `var` variable is not limited to that block.

MDN source: [JavaScript declarations --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let)

2. What is the difference between `==` and `===`? Provide an example where they produce different results.

The == operator is called **loose equality**. It can convert values to compatible types before comparing them. The === operator is called **strict equality**. It compares without converting operands to make their types match; if the types differ, the result is generally `false`. In most situations, === is the clearer and safer choice because it avoids unexpected type conversions.

Example

```
console.log("10" == 10);
console.log("10" === 10);

console.log(0 == false);
console.log(0 === false);
```

Expected console output

```
true
false
true
false
```

What this shows: `"10"` is a string, while `10` is a number. Loose equality converts them for comparison, so `"10" == 10` is `true`. Strict equality does not convert them, so `"10" === 10` is `false`. Similarly, `0 == false` is `true`, but `0 === false` is `false` because a number and a Boolean are different types.

MDN sources: - [Equality (==) --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Equality) - [Strict equality (===) --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Strict_equality)

3. Why does `"10" + 5` return `"105"`, while `"10" - 5` returns `5`?

JavaScript treats the + operator as both numeric addition and string

concatenation. When one operand is a string, the + operator converts the other operand to a string and joins the two strings. Therefore, `"10" + 5` becomes `"10" + "5"`, producing `"105"`.

The `-` operator is used for subtraction, not string concatenation. It converts numeric strings to numbers when possible. Therefore, `"10" - 5` becomes `10 - 5`, producing `5`.

Example

```
console.log("10" + 5);
console.log("10" - 5);

console.log(typeof ("10" + 5));
console.log(typeof ("10" - 5));
```

Expected console output

```
105
5
string
number
```

What this shows: The first expression produces a string, while the second produces a number.

MDN sources: - [Addition (+) --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition) - [Subtraction (`-`) --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Subtraction) - [Expressions and operators --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators)

4. What is the difference between `undefined` and `null`?

Both `undefined` and `null` represent the absence of a useful value, but they are commonly used in different situations.

For example, a declared variable without an initial value has the value `undefined`, and a missing object property also returns `undefined`.

indicate that there is no value or object at that point.

• **`undefined`** usually means that a value has not been assigned.

• **`null`** is a value that a programmer can assign intentionally to

One detail to remember is that `typeof null` returns `"object"`. This is a long-standing behavior in JavaScript. By contrast, `typeof undefined` returns `"undefined"`.

Example

```
let userName;
const selectedProduct = null;

console.log(userName);
console.log(selectedProduct);
```

```
console.log(typeof userName);
console.log(typeof selectedProduct);

console.log(userName == selectedProduct);
console.log(userName === selectedProduct);
```

Expected console output

```
undefined
null
undefined
object
true
false
```

What this shows: `userName` has no assigned value, so it is `undefined`. `selectedProduct` is deliberately set to `null`. Loose equality considers `null` and `undefined` equal to each other, but strict equality does not.

MDN source: [JavaScript language overview --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview)

5. What are truthy and falsy values? List the falsy values and provide examples of truthy values.

JavaScript can treat any value as truthy or falsy when a Boolean condition is expected, such as in an `if` statement.

• **Truthy values** behave like `true` in a Boolean context.

• **Falsy values** behave like `false` in a Boolean context.

The standard falsy values are:

1. `false`
2. `0` and `-0`
3. `0n` (BigInt zero)
4. `""` (an empty string; also `''` and ``)
5. `null`
6. `undefined`
7. `NaN`

All other values are truthy, including `"0"`, `"false"`, `"hello"`, `[]`, and `{}`. A string containing characters is truthy even if it looks like a false value, and empty arrays and empty objects are also truthy.

Example

```
console.log(Boolean(false));
console.log(Boolean(0));
console.log(Boolean(""));
console.log(Boolean(null));
console.log(Boolean(undefined));
console.log(Boolean(NaN));
console.log(Boolean(0n));

console.log(Boolean("hello"));
```

```
console.log(Boolean("0"));
console.log(Boolean([]));
console.log(Boolean({}));

if ("hello") {
    console.log("This message appears because the value is truthy.");
}

if (0) {
    console.log("This message does not appear.");
}
```

Expected console output

```
false
false
false
false
false
false
false
true
true
true
true
This message appears because the value is truthy.
```

What this shows: `Boolean(value)` converts a value to `true` or `false`. In the `if` statements, JavaScript executes the first block because `"hello"` is truthy and skips the second block because `0` is falsy.

MDN sources: - [JavaScript language overview --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview) - [Expressions and operators --- MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators)

How to run the examples in the browser console

Windows, press **F12** or **Ctrl + Shift + J** to open the Console directly.

a multi-line example, paste the whole block and run it.

section.

shown above. Do not run the `blockValue` line in Question 1 unless you want to see the explained `ReferenceError`.

1. Open Chrome, Edge, Firefox, or another modern browser.

2. Open a webpage, then open Developer Tools. In Chrome or Edge on

3. Copy one example at a time into the Console and press **Enter**. For

4. Compare the output with the expected output recorded in each

5. If you run the examples separately, the outputs should match those

Conclusion

Understanding variables, equality operators, type conversion, `null` and `undefined`, and truthy and falsy values helps prevent common JavaScript

mistakes. In modern code, `const` is a good default for values that are not reassigned, `let` is useful for changing values, and === is generally preferred for comparisons. Testing small examples in the browser console makes these rules easier to understand.

References

statement](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let)

(==)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Equality)

(===)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Strict_equality)

(+)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition)

(-)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Subtraction)

operators](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators)

overview](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview)

1. MDN Web Docs. [JavaScript `let`
2. MDN Web Docs. [Equality
3. MDN Web Docs. [Strict equality
4. MDN Web Docs. [Addition
5. MDN Web Docs. [Subtraction
6. MDN Web Docs. [Expressions and
7. MDN Web Docs. [JavaScript language